

Reverse Engineering World of Warcraft

3.3.5a (Build 12340): Lua API and Memory Structures for Automated Quest Interaction

Introduction to Client Execution and Memory Architecture

Automating interactions within the World of Warcraft (WoW) 3.3.5a client (build 12340) requires a sophisticated understanding of the game's execution environment, memory architecture, and the asynchronous, event-driven nature of its user interface. The 3.3.5a client operates as a 32-bit x86 executable, utilizing a customized Lua 5.1 engine to drive the entire user interface and interaction layer.¹ When developing a dynamically linked library (DLL) injected into the client memory space, the most robust method for programmatic interaction bridges direct C++ memory manipulation with the native Lua API.

The client manages game entities through a complex, dynamically allocated data structure known as the Object Manager.² To programmatically interact with a quest-giving Non-Player Character (NPC), the injected architecture must first locate the NPC's Globally Unique Identifier (GUID) within the Object Manager's linked list.² Once the entity is identified and spatial proximity is validated against server-side distance constraints, an interaction is triggered. This is typically achieved by invoking the `InteractUnit` virtual function on the local player object or by calling the Lua equivalent, which transmits a `CMSG_GOSSIP_HELLO` or `CMSG_QUESTGIVER_HELLO` packet to the server.

Because the server holds absolute authority over the game state, the client cannot instantaneously assume an interaction is successful.³ Instead, the injected application must rely on hooking the `FrameScript_Execute` function—the core C++ routine responsible for evaluating Lua strings—to read the subsequent state changes.² Hooking this function allows the DLL to pass raw Lua strings into the engine, effectively bypassing standard UI limitations and capturing the returned data through asynchronous event listeners. Operating at this level allows for the execution of protected and hardware-restricted functions without triggering the client's internal anti-automation mechanisms.

The Object Manager and Programmatic Interaction

Before the Lua API can expose quest data, the client must successfully initiate an interaction with the target unit. The Object Manager maintains a cached list of all objects currently loaded

in the player's immediate environment, including players, items, game objects, and NPCs.²

The execution flow for programmatic interaction begins with parsing this memory structure. The Object Manager is accessed via a static pointer offset relative to the application's base address. By traversing the linked list of objects, the injected code reads dynamic object data—such as health, position coordinates, and status flags—directly from memory.² When a target NPC is identified via its entry ID or GUID, the interaction sequence is initiated.

At the C++ level, triggering an interaction programmatically involves calling the internal game function responsible for right-click emulation. The virtual function `Interact` (commonly found within the `CGameObject` or `CGameUnit` virtual method tables) takes the target's GUID as a parameter.² Alternatively, locating the static address for the `InteractUnit` function requires identifying the C++ function registered to the Lua API. Reverse engineers typically locate this by searching for the "InteractUnit" string literal within the .rdata section of `Wow.exe` and following the cross-references to the API registration block.¹

Executing this function forces the client to calculate the distance to the target; if the target is within the maximum interaction range (typically 5 to 8 yards for quest givers), the client dispatches a network packet to the server requesting interaction. The server validates this request against its own spatial and state data. If validated, the server responds with a packet containing the NPC's current gossip or quest data. It is only at this exact moment that the Lua engine is populated with data, and the corresponding UI events are fired.⁵ The programmatic execution thread must yield control back to the client and wait for these events before proceeding.

Asynchronous Event-Driven State Management

Automated quest interaction cannot be executed linearly as a single block of code. Due to network latency and server-side validation, invoking a function to interact with an NPC and immediately querying the available quests will return null or outdated data. The system must be designed as a finite state machine that transitions based on native Lua events.⁵

The World of Warcraft UI relies on an event dispatcher. When the server transmits quest data, the client processes the packet and fires specific events globally. An injected DLL must register for or monitor these events to know when to proceed to the next step of the interaction flow.

Core Quest Interaction Events

The following table outlines the critical events that govern the lifecycle of NPC quest interaction in client build 12340:

Event Name	Description	State Machine Implication
GOSSIP_SHOW	Fired when the player interacts with an NPC that offers multiple interaction types (e.g., quests, vendor, trainer).	The gossip frame is populated. Data from <code>GetGossipAvailableQuests()</code> and <code>GetGossipActiveQuests()</code> is now safe to query. ⁸
QUEST_GREETING	Fired when an NPC offers quests but lacks standard gossip options, bypassing the generic gossip frame entirely.	Standard gossip functions will return null. The state machine must pivot to using <code>GetAvailableTitle()</code> and <code>GetActiveTitle()</code> equivalent functions. ¹⁰
QUEST_DETAIL	Fired when a specific available quest is selected and its full text is displayed.	The quest detail frame is active. The client has received the quest lore, objectives, and rewards. Safe to call <code>AcceptQuest()</code> . ⁵
QUEST_ACCEPTED	Fired immediately after the server confirms the player has successfully accepted the quest.	The quest is now in the player's quest log. The state machine can close the interaction or pick up additional quests. ⁵
QUEST_PROGRESS	Fired when an active (ongoing/completable) quest is selected from the NPC, showing the turn-in requirements.	The NPC asks for required items or objectives. Safe to evaluate if the quest is complete and call <code>CompleteQuest()</code> . ⁹
QUEST_COMPLETE	Fired after <code>CompleteQuest()</code> is called, transitioning to the final reward selection screen.	Safe to evaluate reward choices and finalize the transaction by calling <code>GetQuestReward(choiceIndex)</code> . ⁵
QUEST_FINISHED	Fired when the interaction frame changes state or	Acts as a cleanup event to reset the automation state

	closes completely.	machine, indicating the interaction window has terminated. ⁵
--	--------------------	---

Understanding this asynchronous flow is paramount. A standard quest pickup sequence dictates the following state transitions: InteractUnit -> Wait for GOSSIP_SHOW -> Call SelectGossipAvailableQuest(index) -> Wait for QUEST_DETAIL -> Call AcceptQuest() -> Wait for QUEST_ACCEPTED.⁵ Deviating from this sequence or ignoring the event delays will result in dropped actions and potential server desynchronization.

Extracting Quest Data: Available vs. Active Quests

Once the GOSSIP_SHOW event has fired, the client memory is populated with lists of quests offered by the current NPC. The player fundamentally must have the gossip window open (i.e., successfully interacted with the NPC) for these Lua functions to return valid data, as the client strictly binds this data to the active interaction context.⁹ The Lua API bifurcates these lists into two distinct categories: available quests (quests the player can pick up) and active quests (quests the player is currently on, which the NPC is involved with).¹¹

Available Quests Parsing

To retrieve quests that are available to be accepted, the API provides two primary functions: GetNumGossipAvailableQuests() and GetGossipAvailableQuests().⁸ The former returns a simple integer indicating the total count of available quests, which is highly useful for initializing iteration loops.¹² The latter retrieves the actual quest metadata.

Because Lua naturally handles multiple return values natively via stack pushes, the return format of GetGossipAvailableQuests() in build 12340 is a flattened, dynamically sized list. It provides exactly five specific variables per quest, repeated consecutively for however many quests exist.⁸

Return Format (per quest):

title, level, isLowLevel, isDaily, isRepeatable

If an NPC offers two quests, the function returns ten variables onto the Lua stack: title1, level1, isLowLevel1, isDaily1, isRepeatable1, title2, level2, isLowLevel2, isDaily2, isRepeatable2.⁸

Parameter	Type	Description
title	String	The localized name of the

		quest. ⁸
level	Number	The intended level of the quest. ⁸
isLowLevel	Boolean/Number	Returns 1 if the quest is considered trivial (gray difficulty) to the player, nil otherwise. ⁸
isDaily	Boolean/Number	Returns 1 if the quest is a daily quest (marked by a blue exclamation mark), nil otherwise. ⁸
isRepeatable	Boolean/Number	Returns 1 if the quest is infinitely repeatable, nil otherwise. ⁸

Note regarding version discrepancies: In later expansions (e.g., Mists of Pandaria and Legion), this API was updated to return additional fields such as isLegendary and isIgnored.¹³ In 3.3.5a (WotLK), the return signature strictly adheres to the five parameters outlined above. Automation logic must not expect the expanded parameters, or stack alignment errors will occur.

To iterate through this flattened list in a Lua script executed via a FrameScript_Execute hook, the code must dynamically group the returns by five:

Lua

```

local quests = {GetGossipAvailableQuests()}
local availableCount = GetNumGossipAvailableQuests()
for i = 1, availableCount do
    local offset = (i - 1) * 5
    local title = quests[offset + 1]
    local level = quests[offset + 2]
    local isLowLevel = quests[offset + 3]
    local isDaily = quests[offset + 4]
    local isRepeatable = quests[offset + 5]
    -- Process quest acceptance logic here

```

end

Active Quests Parsing

To distinguish between a quest that is available to pick up and a quest that is ready to turn in, the injected architecture must query the active quests list. Active quests are those the player already possesses in their quest log that correspond to the targeted NPC.⁹

The API provides `GetNumGossipActiveQuests()` to return the total count, and `GetGossipActiveQuests()` to retrieve the metadata.⁹ The return format in 3.3.5a differs slightly from available quests, providing exactly four variables per quest in a flattened array.⁹

Return Format (per quest):

title, level, isLowLevel, isComplete

Parameter	Type	Description
title	String	The localized name of the active quest. ⁹
level	Number	The intended level of the quest. This frequently returns -1 if the quest involves non-standard level scaling or specific profession requirements. ⁹
isLowLevel	Boolean/Number	Returns 1 if trivial, nil otherwise. ⁹
isComplete	Boolean/Number	Returns 1 if all objectives are met and the quest is ready for turn-in (indicated visually by a yellow question mark), nil otherwise (gray question mark). ⁹

Automation logic evaluating turn-ins must parse this list, step through the returns by a factor of four, and strictly verify that `isComplete` evaluates to 1 before attempting to select the quest for completion.⁹ If `isComplete` returns nil, the NPC is merely involved in an ongoing quest step, and attempting to force a completion sequence will result in a server-side rejection.

The Quest Acceptance Protocol

Once the target quest is identified from the available list, the state machine must transition from evaluating gossip data to initiating the acceptance sequence.

The function `SelectGossipAvailableQuest(index)` is invoked to choose the quest.¹⁴ The index parameter is a strictly 1-based integer corresponding to the ordinal position of the quest in the list returned by `GetGossipAvailableQuests()`.¹⁴ For instance, to select the very first quest offered by the NPC, the automation must pass 1. The client relies on this index rather than a quest ID because the gossip window's contextual memory is ordered sequentially.

Executing this function triggers the client to construct and send a network payload to the server (`CMSG_GOSSIP_SELECT_OPTION`), indicating the player's intent to view the specific quest's details. The server validates this request and responds with `SMSG_QUESTGIVER_QUEST_DETAILS`, a packet containing the lore text, completion objectives, and reward structures. Upon receiving and parsing this packet, the client populates the Quest Detail frame in memory and fires the `QUEST_DETAIL` Lua event.⁵

The injected automation DLL must intercept the `QUEST_DETAIL` event. At this precise moment, the quest details are fully loaded into the client's localized memory context. The programmatic sequence then calls the `AcceptQuest()` Lua function.¹⁶ Executing `AcceptQuest()` constructs and transmits the `CMSG_QUESTGIVER_ACCEPT_QUEST` packet to the server, formally requesting the quest be added to the player's database record.

The server performs a final, rigid validation sequence. It verifies the player's current level against the quest's prerequisites, confirms previous quests in the chain have been completed, and ensures the player's quest log has not reached its 25-quest maximum capacity. If successful, the server adds the quest to the character's record and responds with a confirmation packet. This prompts the client to fire the `QUEST_ACCEPTED` event.⁵

The payload of the `QUEST_ACCEPTED` event (`arg1`) contains the newly generated quest log index.⁵ This integer can be immediately captured by the event listener and passed to `GetQuestLogTitle(index)` to extract the localized name and internal questID for external logging and progression tracking.⁵ Once `QUEST_ACCEPTED` fires, the state machine can safely close the interaction frame or loop back to accept additional quests.

Reward Memory Structures and ItemLink Parsing

Evaluating the economic and character progression value of a quest before acceptance, or selecting the optimal reward during turn-in, requires reading the reward structures mapped into client memory during the `QUEST_DETAIL` or `QUEST_COMPLETE` states. The client segregates rewards into unconditional rewards (items the player is guaranteed to receive automatically) and choice rewards (where the player must select one item from a list to finalize

the quest).¹²

The API exposes counts for these categories via `GetNumQuestRewards()` for mandatory items and `GetNumQuestChoices()` for selectable options.¹² Similarly, abstract rewards are extracted via `GetQuestMoneyReward()` (or `GetRewardMoney()`), which returns the monetary reward in copper coins, and experience yields via `GetQuestXPReward()`, which calculates the experience based on the player's current level relative to the quest level.¹⁹ Spell-based rewards, such as reputation gains or auras, can be extracted using `GetRewardSpell()`.¹⁹

Item Information and Hyperlink Parsing

Extracting detailed item information requires querying the underlying item cache using the quest interaction API. The function `GetQuestItemInfo("type", index)` returns basic string and numeric data about the reward, such as its name, texture path, quantity, quality, and usability status.¹² However, this function is insufficient for obtaining the raw `itemID` needed for database cross-referencing.

To retrieve the exact `itemID`, the automation must invoke `GetQuestItemLink("type", index)`.²⁰ The "type" parameter must be a string: "choice" (for selectable rewards), "reward" (for guaranteed rewards), or "required" (for items required to turn in the quest).²⁰ The index is the 1-based position of the item.

The return value is an `ItemLink`, a heavily structured string format utilized by the WoW client to embed metadata directly into chat streams and UI tooltips.¹² To make automated decisions based on item IDs (e.g., querying a local SQLite database for vendor value, auction house prices, or stat weights), the injected DLL must parse this complex string.¹²

The standard `ItemLink` format in WotLK 3.3.5a follows this structure:

`|cffRRGGBB|Hitem:itemID:enchant:gem1:gem2:gem3:gem4:suffix:unique:linkLvl:name|h|h|r`.¹²

Component	Description
cffRRGGBB	The hexadecimal color code corresponding to item quality (e.g., ff0070dd for Rare/Blue, ffa335ee for Epic/Purple). ¹²
Hitem:	The hyperlink protocol identifier indicating the start of the item string data. ¹²
itemID	The primary numerical identifier of the item as it exists in the server's <code>item_template</code>

	database. ¹²
enchant:gem1...	Subsequent numerical fields separated by colons representing dynamic modifications to the item (often zeroes for base quest rewards). ¹²
h h	The localized, human-readable text enclosed in brackets that is rendered in the UI. ¹²
r	The color reset escape sequence, restoring the text to the default chat color. ¹²

Extracting the exact itemID requires executing a regular expression within the Lua environment or passing the raw string back to the C++ memory space for tokenization. Within Lua, the standard string.match implementation achieves this efficiently:

Lua

```
local itemLink = GetQuestItemLink("choice", 1)
if itemLink then
    local itemID = string.match(itemLink, "item:(%d+)")
    -- Proceed with database evaluation using the numeric itemID
end
```

This pattern matches the literal string "item:" and captures the subsequent sequence of numeric digits, extracting the exact ID necessary for programmatic evaluation.¹²

It is critical to note that the client resolves these ItemLinks by querying its local memory cache (WDB files). Occasionally, the client's local item cache may not possess the data for a newly introduced item or an item cleared from memory. In such cases, GetQuestItemLink will return nil or a partial item ID without the localized name. When this occurs, the automation must either force a cache update by querying the server (Item:CreateFromItemID equivalent in older APIs) and wait for an asynchronous cache update, or implement retry logic in the state machine before proceeding.²¹

The Quest Completion and Turn-In Sequence

The protocol for completing and turning in a quest mirrors the complexity of the acceptance

flow but involves more critical state transitions, especially when reward selection is mandated by the server.

1. **Active Quest Selection:** Upon receiving GOSSIP_SHOW, the automation iterates over the data returned by `GetGossipActiveQuests()`. If a quest evaluates with `isComplete == 1`, the DLL executes `SelectGossipActiveQuest(index)`, passing the 1-based index corresponding to the quest's position in the active list.⁹
2. **Progress Evaluation:** The client transmits a `CMSG_GOSSIP_SELECT_OPTION` packet to the server and receives the quest progress details. The `QUEST_PROGRESS` event fires.⁹ At this stage, the NPC's dialogue updates, typically demanding the required items. The automation must now invoke the `CompleteQuest()` Lua function.¹⁶
3. **Completion Dialogue Transition:** Executing `CompleteQuest()` bridges the progress state to the final reward state. It signals to the server that the player wishes to hand over the items. The server responds with the reward data payload, and the client subsequently fires the `QUEST_COMPLETE` event.⁵
4. **Reward Finalization:** Within the `QUEST_COMPLETE` event listener, the automation must evaluate if a choice is required to finalize the transaction. If `GetNumQuestChoices() > 0`, the logic must iterate through the choices, evaluate the optimal item index using the parsed `ItemLinks`, and execute the final turn-in via `GetQuestReward(choiceIndex)`.¹² The `choiceIndex` is a 1-based integer corresponding to the selected reward. If no choice is required, the function is conventionally executed with a 0 or 1 argument (or called without arguments depending on UI bounds), finalizing the transaction.²⁶
5. **Confirmation and Cleanup:** The server validates the transaction, removes the required quest items from the player's inventory, credits the experience, money, and reward items, and removes the quest from the quest log. The client then fires `QUEST_FINISHED`, signaling the interaction frame has closed, allowing the automation to reset its internal state machine.⁵

Quest Log Memory and State Management

Maintaining an accurate internal representation of the player's quest log is vital. It prevents the automation from attempting to accept quests it already possesses, prevents routing to turn-in NPCs for incomplete quests, and allows for the calculation of spatial routing paths based on active objectives.

The quest log in build 12340 is queried using `GetNumQuestLogEntries()` to determine the total number of lines currently rendered in the log.²⁷ It is critical to understand that the quest log API evaluates the UI state, meaning the total entries include both actual quests and the collapsible zone headers (e.g., "Elwynn Forest", "Icecrown").²⁸

Iteration requires stepping from 1 to `GetNumQuestLogEntries()` and evaluating `GetQuestLogTitle(index)` for each iteration.¹⁹ In 3.3.5a, this function returns a verbose set of parameters: title, level, questTag, suggestedGroup, isHeader, isCollapsed, isComplete, isDaily,

questID.²⁹

Parameter	Type	Description and Automation Implication
title	String	Localized name of the quest. ²⁹ Used for legacy string matching if IDs are unavailable.
isHeader	Boolean/Number	Returns 1 if the entry is a zone grouping header, nil otherwise. ²⁹ Critical: Automation must evaluate this flag and skip iteration loops where isHeader == 1 to prevent logic errors and null pointer exceptions. ²⁸
isComplete	Number	Returns 1 (Complete), -1 (Failed), or nil (In Progress). ²⁹ Dictates whether to route the player to the turn-in NPC or continue grinding/looting.
questID	Number	The unique numerical database ID. ²⁹ (Note: Added in patch 3.3.0). Used for precise database cross-referencing, avoiding localization string matching issues entirely.

While modern expansions include functions like `GetQuestLogIndexByID()`, in standard WoTLK clients, obtaining the log index for a specific quest requires a manual iteration loop matching the `questID` returned from `GetQuestLogTitle(index)`. By traversing the log and capturing the `questID` and `isComplete` status of every non-header entry, the injected DLL maintains a real-time, highly accurate replica of the player's progression state in its C++ memory space.

Server-Side Validation and TrinityCore Architecture

When operating on private servers based on the TrinityCore or AzerothCore architecture, the automation must strictly adhere to the emulation constraints defined by the server's relational database.³ Even if a modified client manipulates memory to display a quest as available and transmits a CMSG_QUESTGIVER_ACCEPT_QUEST packet, the server will silently drop or explicitly reject interaction packets that violate database constraints.

The server relies heavily on the quest_template table to validate all quest-related network traffic.³ Key fields evaluated during the acceptance and turn-in phases include:

- **MinLevel & MaxLevel:** Absolute level boundaries enforced by the server.³ If the player's level falls outside this range, the packet is dropped.
- **RequiredClasses & AllowableRaces:** Bitmasks defining demographic restrictions.³ For example, an AllowableRaces bitmask of 690 (calculating the bits for Orc, Undead, Tauren, Troll, and Blood Elf) restricts the quest exclusively to the Horde faction. A client spoofing this request will be ignored.³
- **QuestType:** Dictates whether the quest auto-completes upon acceptance (Type 0) or requires standard objective completion (Type 2).³
- **Distance Checks:** The server continuously measures the 3D coordinate distance between the player's GUID and the NPC's GUID upon receiving any interaction packet. Triggering Lua functions when the entity is strictly out of range results in a silent packet drop, and repeated violations frequently trigger server-side heuristic anti-cheat flags. Automation must mathematically verify the Euclidean distance between the CGPlayer and CGUnit vectors before invoking InteractUnit.

Client-Side Protections, Hardware Taint, and Anti-Cheat Evasion

The World of Warcraft client employs a robust security mechanism known as the Taint system to prevent automated UI code from executing sensitive actions.¹ Functions classified as PROTECTED (e.g., movement APIs, spellcasting) can only be invoked by code digitally signed by Blizzard. A subset of functions is classified as HW (Hardware Event required), meaning they can only execute in immediate response to a physical keyboard or mouse input from the user (e.g., within an OnClick handler).¹

In the context of the quest API for build 12340, functions such as SelectGossipAvailableQuest, AcceptQuest, and CompleteQuest are generally considered unprotected relative to strict combat APIs.¹ However, invoking them through standard macro /script commands at high velocities without user interaction can still trigger heuristic taint warnings, leading to UI errors.

When injecting a C++ DLL, executing Lua via FrameScript_Execute inherently sidesteps standard UI taint because the execution originates from the C++ environment itself, operating at the engine level rather than the addon sandbox.¹ Nevertheless, if a specific API operation is

updated or restricted to demand a strict hardware event validation, the engine checks an internal state flag (often populated by the message loop processing WM_KEYDOWN or WM_LBUTTONDOWN messages) before allowing the execution.

Bypassing Hardware Event Restrictions in Memory

To completely eradicate hardware event restrictions for programmatic automation, reverse engineers utilize disassemblers like IDA Pro to locate the client's internal validation routines.¹

The Lua engine implements a C++ function that evaluates the hardware state. By locating the cross-references to the error string *"A macro script has been blocked from an action only available to the Blizzard UI"*, an engineer can trace back to the exact conditional jump (jz or jnz) determining the hardware state.¹ Patching this instruction in memory—for example, replacing a jz (Jump if Zero) with a jmp (Unconditional Jump) or filling the constraint check with NOP (No Operation, 0x90) instructions—forces the client to globally assume a hardware event is always present, thereby permanently unlocking the API.¹

Anti-Cheat Considerations (Warden)

Modifying the .text segment of Wow.exe to patch hardware events via byte replacement is highly detectable by Warden, Blizzard's proprietary anti-cheat engine, which is frequently adapted and implemented on advanced 3.3.5a private servers.⁴ Warden periodically requests hashes of specific memory pages. If the hardware validation routine is patched with NOP instructions, the memory hash will mismatch the server's expected value, resulting in an immediate account ban.

To circumvent this, sophisticated DLL implementations avoid direct byte patching. Instead, they employ Vectored Exception Handling (VEH) combined with Hardware Breakpoints utilizing the CPU's Debug Registers (DR0-DR3). By setting a hardware breakpoint on the hardware validation function, the CPU triggers an EXCEPTION_SINGLE_STEP before the validation instruction executes. The injected VEH handler intercepts the exception, artificially manipulates the EAX or EFLAGS registers to spoof a positive hardware state, and resumes execution.⁴ This leaves the actual memory bytes untouched and entirely transparent to Warden's hashing algorithms, ensuring secure, undetected execution of all quest interaction APIs.

Conclusion

Mastering automated quest interaction in World of Warcraft 3.3.5a demands a flawless synthesis of C++ memory manipulation and deep Lua API orchestration. Because the client acts merely as a visual proxy for the server's authoritative state machine, executing linear commands without respecting network latency and event dispatching (GOSSIP_SHOW, QUEST_DETAIL, QUEST_COMPLETE) will cause systemic failures. By rigorously hooking FrameScript_Execute, correctly parsing the dynamic return formats of GetGossipAvailableQuests and GetGossipActiveQuests, extracting precise ItemLinks via regex

pattern matching, and utilizing stealthy memory techniques like VEH to bypass hardware event validations, an injected architecture can perfectly replicate complex human-NPC interactions within the strict bounds of the engine's architecture. Utilizing this comprehensive methodology ensures stable, undetectable automation aligned with both client constraints and server-side emulation mechanics.

Works cited

1. Unlocking WoW-API Functions in 3.3.5a using Disassembler - Roman Hergenreder, accessed March 4, 2026, <https://romanh.de/article/Unlocking-API-Functions-in-WoW-335a-using-a-Disassembler>
2. AzDeltaQQ/WotLKRotations: A Python-based experimental framework for interacting with World of Warcraft (3.3.5a - 12340 client) memory to monitor game state and potentially execute combat rotations. (Extremely Work-in-progress) - GitHub, accessed March 4, 2026, <https://github.com/AzDeltaQQ/WotLKRotations>
3. quest_template - TrinityCore - Confluence, accessed March 4, 2026, https://trinitycore.atlassian.net/wiki/display/tc/quest_template
4. How I documented all CVar values in WoW 3.3.5.12340 - Function Hooking with C++ & MS Detours : r/ReverseEngineering - Reddit, accessed March 4, 2026, https://www.reddit.com/r/ReverseEngineering/comments/14otibo/how_i_documented_all_cvar_values_in_wow_33512340/
5. WoW:Events/Quest - AddOn Studio, accessed March 4, 2026, <https://addonstudio.org/wiki/WoW:Events/Quest>
6. GetGossipActiveQuests - Warcraft Wiki - Your wiki guide to the World of Warcraft, accessed March 4, 2026, https://warcraft.wiki.gg/wiki/API_GetGossipActiveQuests
7. wow-ui-source/FrameXML/GossipFrame.lua at master - GitHub, accessed March 4, 2026, <https://github.com/Ennie/wow-ui-source/blob/master/FrameXML/GossipFrame.lua>
8. API GetGossipAvailableQuests - WoWWiki - Fandom, accessed March 4, 2026, https://wowwiki-archive.fandom.com/wiki/API_GetGossipAvailableQuests
9. API GetGossipActiveQuests - WoWWiki - Fandom, accessed March 4, 2026, https://wowwiki-archive.fandom.com/wiki/API_GetGossipActiveQuests
10. [help] QUEST_GREETING event : r/wowaddons - Reddit, accessed March 4, 2026, https://www.reddit.com/r/wowaddons/comments/ki0npq/help_quest_greeting_event/
11. WoW lua: Getting quest attributes before the QUEST_DETAIL event, accessed March 4, 2026, <https://gamedev.stackexchange.com/questions/8058/wow-lua-getting-quest-attributes-before-the-quest-detail-event>
12. WoW API: World of Warcraft API - AddOn Studio, accessed March 4, 2026, https://addonstudio.org/wiki/WoW:World_of_Warcraft_API

13. GetGossipAvailableQuests - Warcraft Wiki, accessed March 4, 2026,
https://warcraft.wiki.gg/wiki/API_GetGossipAvailableQuests
14. API SelectGossipAvailableQuest - WoWWiki - Fandom, accessed March 4, 2026,
https://wowwiki-archive.fandom.com/wiki/API_SelectGossipAvailableQuest
15. SelectGossipAvailableQuest - Wowpedia - Your wiki guide to the World of Warcraft, accessed March 4, 2026,
https://wowpedia.fandom.com/wiki/API_SelectGossipAvailableQuest
16. [SOLVED] Quest Select/Accept/Complete Macro - Blizzard Forums, accessed March 4, 2026,
<https://eu.forums.blizzard.com/en/wow/t/solved-quest-selectacceptcomplete-macro/397992>
17. XP BAR v1.0.0 WOTLK-WEAKAURA - Wago.io, accessed March 4, 2026,
<https://wago.io/r-TeP9Lrc>
18. World of Warcraft API, accessed March 4, 2026,
https://warcraft.wiki.gg/wiki/World_of_Warcraft_API
19. 魔兽世界API魔兽世界全局函数转载 - CSDN博客, accessed March 4, 2026,
https://blog.csdn.net/qq_18882253/article/details/117822181
20. API GetQuestItemInfo - WoWWiki - Fandom, accessed March 4, 2026,
https://wowwiki-archive.fandom.com/wiki/API_GetQuestItemInfo
21. API GetQuestLogItemLink - WoWWiki - Fandom, accessed March 4, 2026,
https://wowwiki-archive.fandom.com/wiki/API_GetQuestLogItemLink
22. GetQuestItemLink - Warcraft Wiki - Your wiki guide to the World of Warcraft, accessed March 4, 2026, https://warcraft.wiki.gg/wiki/API_GetQuestItemLink
23. WoW API type: ItemString - AddOn Studio, accessed March 4, 2026,
<https://addonstudio.org/wiki/WoW:ItemString>
24. Lua Patterns - World of Warcraft Vanilla - Stack Overflow, accessed March 4, 2026,
<https://stackoverflow.com/questions/42168811/lua-patterns-world-of-warcraft-vanilla>
25. QuestLogSpecialItem question - UI and Macro - World of Warcraft Forums, accessed March 4, 2026,
<https://us.forums.blizzard.com/en/wow/t/questlogspecialitem-question/2164155>
26. API d_of_Warcraft_API | Khazaripedia Wiki - Fandom, accessed March 4, 2026,
https://khazaripedia.fandom.com/wiki/World_of_Warcraft_API
27. It's about time. End of expansion. Here's a macro to clear out entire quest log so you go in fresh. : r/wow - Reddit, accessed March 4, 2026,
https://www.reddit.com/r/wow/comments/1ren37y/its_about_time_end_of_expansion_heres_a_macro_to/
28. How to get the name of a specific quest in the Quest Log in WoW? - Stack Overflow, accessed March 4, 2026,
<https://stackoverflow.com/questions/66589920/how-to-get-the-name-of-a-specific-quest-in-the-quest-log-in-wow>
29. API GetQuestLogTitle - WoWWiki - Fandom, accessed March 4, 2026,
https://wowwiki-archive.fandom.com/wiki/API_GetQuestLogTitle
30. Category:World of Warcraft API/Protected Functions - WoWWiki, accessed

March 4, 2026,

https://wowwiki-archive.fandom.com/wiki/Category:World_of_Warcraft_API/Protected_Functions

31. WoW API: Global functions - AddOn Studio, accessed March 4, 2026,
https://addonstudio.org/wiki/WoW:Global_functions
32. A small patcher to bypass protected Lua functions in World of Warcraft 3.3.5a client `Wow.exe` - GitHub Gist, accessed March 4, 2026,
<https://gist.github.com/trevor403/90363a9edafd19094d844b1fbfdbb76e>